

The machine learning part

Status: one of four models built. The rest are designed, not written. **Specification:** §7.2, with §7.1 as the reports it sits beside, and §14 for the schedule. **Last updated:** 5 October 2026.

How to use this file

This is the briefing for whoever builds the rest of the machine learning. It is written so that you can start from here: what the models are, what they need, what they give the shop owner, what they take away from them when they are wrong, and the traps that are specific to this application rather than to machine learning in general.

Read it in order the first time. After that, the feature sections stand alone.

If you only read one section, read **Why three months of data** and **The difficulties, honestly**. Everything else is engineering. Those two are the reasons this part of the project is hard, and the reasons a model here can do harm rather than nothing.

1. What "the ML part" actually is

Four separate things, all living in `backend/app/ml/`. They are not one model with four outputs, and they do not share training data in any interesting way. They share only a principle: each replaces a rule that is currently a fixed number or a fixed formula with something learned from this shop's own history.

#	Model	Replaces	File	Built?
1	Demand forecast and run-out date	nothing — new capability	<code>ml/forecast.py</code>	yes
2	Smarter low-stock threshold	<code>domain/thresholds.suggest_from_velocity</code> (flat average)	<code>ml/threshold_suggestion.py</code>	stub
3	Learned festival uplift	<code>seasonal_events.expected_uplift_pct</code> (one number for every shop)	<code>ml/seasonal_uplift.py</code>	stub
4	Supplier recommendation from outcomes	<code>domain/ranking.py</code> (fixed weights 40/30/20/10)	<code>ml/supplier_recommendation.py</code>	stub

Each stub already has the right signature, a docstring explaining what it will do, and an `is_available()` that returns `False`. They are interfaces, not placeholders to be deleted.

What is deliberately *not* machine learning, and must not become it:

- `domain/seasonal.py` — the date comparison that decides whether a festival card shows. §6.2 calls for a date comparison and it must stay one. `docs/08-api-contract.md` is explicit: the seasonal warning "is not machine learning, and it must not become machine learning". The learned version is model 3, a separate file, so the dashboard does not silently change behaviour when it lands.

- `domain/ranking.py` — the weighted supplier score. §12 states it does not use machine learning. Model 4 is an addition beside it, not a replacement inside it.
- `domain/stock.classify` — in stock / low stock / restock requested. A threshold comparison.

The pattern is worth keeping: **a learned version of a rule goes in a new file in `ml/`, and the plain rule stays where it is.** That way a model that turns out to be wrong can be switched off by one `is_available()` returning `False`, and the application still works exactly as it did.

2. What is already built

`backend/app/ml/forecast.py`, 32 tests in `backend/tests/test_ml.py`.

Pure arithmetic — no database, no network, no imports from the feeds layer. It takes rows and returns a dataclass. That is why it can be tested without a database, which is the convention in `CLAUDE.md`: rules go in `domain/` and `ml/`, fetching stays in `service.py`.

What it computes, per product:

- `units_per_day` — an exponentially weighted mean with a 14-day half-life. A sale a fortnight ago counts half as much as one today.
- `days_of_cover` — `quantity_on_hand / units_per_day`.
- `runs_out_on` — today plus the cover, suppressed beyond 90 days because "runs out in fourteen months" is arithmetic pretending to be a forecast.
- `trend` — rising, steady or falling, from the last 7 days against the 7 before. Needs a fortnight, and a 10% change, before it will claim a direction.
- `suggested_threshold` — delegates to `domain/thresholds.suggest_from_velocity`, deliberately, so the Reports screen and the Stocks screen never show two different suggestions.
- `threshold_looks_wrong` — whether the level the shop set is 50% out either way.

Shop-level:

- `confidence` — `rough` under 28 days, `fair` at 28, `good` at 56 (\$7.2's own eight-week bar).
- `weeks_until_good` — how many more weeks before the number is worth trusting.
- `has_enough` — `False` under 7 days of history, and then the report returns no products at all.

Two decisions inside it worth knowing, because they are the ones an interviewer would poke at:

No scikit-learn, no numpy. Both are commented out in `requirements.txt` and should stay that way until model 3. A linear regression or an ARIMA fitted through thirty daily points will explain the past almost perfectly and predict noise, and it will do so with an R^2 that looks like success. The honest model for this much data is a weighted average. The weighting is the only cleverness, and it is the right cleverness: it answers "is this product moving *now*", which is the question a shop owner is actually asking.

The denominator is the span of history, not the number of days that had a sale. A shop open twenty days that sold rice on four of them sells rice slowly. Dividing by four would call it a fast mover and reorder five times too much. This is the single easiest way to get a velocity badly wrong and it is a one-character mistake.

3. The data it learns from

Everything comes from `sales_records`, which is the training table for every model here. It was built in migration 0009 for exactly this purpose.

```
sales_records
  id          uuid
  upload_id    uuid null  -> sales_uploads(id)  -- set when source = 'upload'
  pos_sale_id  uuid null  -> pos_sales(id)      -- set when source = 'pos'
  source       text       -- 'upload' | 'pos'
  stock_item_id  uuid null  -> stock_items(id)
  catalog_product_id  uuid null  -> product_catalog(id)
  quantity_sold integer    -- may be NEGATIVE: a return (migration 0027)
  sale_date     date       -- the shop's local date
  created_at, updated_at
```

Two sources, one table. A row comes either from a spreadsheet the shop uploaded (`feeds/customer/uploads/applier.py`) or from a till sale (`feeds/customer/pos/service.py` , written with the service key). Reports count them identically, which is the point.

The trap that cost a day: ownership is reached through the parent

`sales_records` has **no owner column**. Ownership is reached through `upload_id` or `pos_sale_id`. The policy `sales_records_own` originally checked only `upload_id`, and a till sale has none — so every POS row was written successfully by the service key and was then **invisible to the shop that rang it**. Nothing errored. `exists (... where u.id = null)` is false, not an error, so a nullable foreign key inside a policy is a silent deny.

Reports read this table through the **owner's** token, so a shop billing entirely through the till was told it had no sales history while sitting on hundreds of rows.

Fixed by **migration 0032**, which adds the `pos_sales` branch. *If you are reading this and reports still show nothing, check 0032 has been run.*

The general lesson, because it will happen again: **when you add a second parent to a table, revisit every policy that reaches ownership through the first one.**

What `sales_records` does *not* record, and what that costs

Missing	Why it matters	Where it could come from
Price per line	Demand depends on price. A model cannot tell a surge from a discount.	<code>pos_sale_items.unit_price</code> exists — join it, or add a price column.
Whether the product was in stock that day	See censored demand in §7. This is the most damaging gap.	Reconstruct from <code>stock_adjustments</code> , which records every change.
Time of day	A weekday-and-hour model would want it.	<code>pos_sales.sold_at</code> is a timestampz.
Why nothing sold	A closed shop and a day with no customers look identical.	Nowhere. Needs a "shop was closed" concept.
Promotions	A one-off discount reads as real demand and inflates the forecast.	Nowhere. Would need a new table.

Note that only one of those is actually unavailable. The others are joins nobody has written yet.

4. Where the data stands today

Measured against the live Supabase project on **3 October 2026**:

- **37 rows** in `sales_records` — 13 from uploads, 24 from the till
- **7 distinct days**, spanning 2026-09-10 to 2026-10-03
- **9 distinct products**
- **1 shop** with any history at all

For scale: `forecast.py` reports `confidence = "rough"` and `weeks_until_good = 7`.

And a harder problem than the volume: this is not real data. Those 37 rows were created while testing the application — sales rung to check the till worked, uploads run to check the parser worked. A model trained on them learns the team's testing habits, not a grocery shop's trade. Three zero-value bills in the list are from testing the price prompt.

The whole team shares one Supabase project (`CLAUDE.md`). So the training table contains everybody's test data, and there is no separation between "data that reflects a shop" and "data that reflects a developer pressing buttons". Any serious evaluation of these models needs either a real shop using the app for a season, or a clearly-labelled generated dataset.

5. Why three months of data

This is the question to be able to answer properly, because the honest answer is not "more is better" — it is that each model needs a specific number of *repeats of a cycle*, and the cycles have different lengths.

The weekly cycle needs 8 to 12 repeats

A grocery shop's week is not flat. Saturday is not Tuesday; pay-day week is not the week after. To separate "Saturdays are busy" from "that Saturday happened to be busy", you need enough Saturdays to average over. The rough statistical rule is that estimating a per-weekday effect reliably wants **at least 8, comfortably 12, observations of each weekday** — so **8 to 12 weeks**, which is where \$7.2's eight-week bar comes from and why `MIN_WEEKS_OF_HISTORY = 8`.

With 7 days you have exactly one of each weekday. The "weekday effect" you would measure is the noise of a single day, and the model would carry it forward confidently.

Variance needs more than the average does

An average stabilises quickly; the *spread* around it does not. This matters because a reorder point is not driven by average demand — it is driven by how bad a bad week can be. Two products both selling 5 a day are different businesses if one sells exactly 5 every day and the other sells 0 for six days and 35 on Sunday. The second needs a much higher threshold to avoid an empty shelf, and you cannot see the difference until you have watched several weeks.

This is precisely what model 2 adds over the flat average, and it is why model 2 is *not* worth building before the data exists: with three weeks it would produce a variance estimate less reliable than the average it replaces.

A festival needs a full cycle, which is a year

Model 3 learns how much a shop's sales rise before Avurudu by comparing its sales around past Avurudus against its baseline. **With one Avurudu you have one observation and no idea whether it was typical.** There is no statistical shortcut: you cannot learn an annual pattern from three months, however much data those three months contain.

Three months is enough for models 1 and 2. Model 3 needs a year, and model 4 needs enough *completed orders* rather than enough days.

So what does "three months" buy, concretely

Horizon	What becomes possible
1 week	A velocity and a run-out date, labelled <code>rough</code> . Already works.
4 weeks	<code>fair</code> . A trend that means something. Worst-sellers are real.
8 weeks	<code>good</code> , \$7.2's bar. Weekday effects become measurable. Model 2 is worth building.
12 weeks (3 months)	Variance estimates are stable. Reorder points can be trusted. Seasonal <i>drift</i> is visible even without a festival.
1 year	Model 3 becomes possible at all.

A note on what the numbers cannot fix

A shop with 36 products and 20 sales a day gives you roughly **0.5 sales per product per day**. For a slow product — one that sells a few units a month — three months is a handful of data points no matter how long the overall history is. Per-product forecasts will stay `rough` for most of the catalog for a long time, and the right response is to say so rather than to average across products and pretend.

6. The four models in detail

Model 1 — Demand forecast and run-out date built

What the owner gets. "You will run out of Sunlight soap on Thursday." A list sorted by what runs out soonest, with how fast each product is moving and whether that is rising or falling.

Before this existed. The Stocks screen showed a quantity and a status badge. "Low stock" told the owner the shelf was thin; it did not tell them *when* it empties, so there was no way to tell a product that needs ordering today from one that is fine for a fortnight. Ordering was by eye and by memory.

Needs. 7 days minimum, 8 weeks to be trustworthy. `stock_items.quantity_on_hand` for the cover calculation. Supplier lead times for the threshold suggestion.

Still to do on it:

- Exclude stock-out days from the rate (see **censored demand**). This is the biggest remaining correctness issue in model 1.
- A weekday effect, once there are 8+ weeks.
- Consider joining price, so a promotion can be identified rather than learned from.

Model 2 — Smarter low-stock threshold

What the owner gets. A suggested `low_threshold` per product that accounts for how *erratic* demand is, not just how high. A product selling 5 a day steadily gets a tight threshold; one averaging 5 a day in weekly spikes gets a higher one, because the reorder point has to survive the spike.

What exists now. `domain/thresholds.suggest_from_velocity(units, days, lead_time)` — a flat average over the history, covering `DEFAULT_COVER_DAYS = 7` plus the supplier's lead time. It refuses below 14 days of history. §6.7 is explicit that this does not need to be machine learning, and it is genuinely useful as it stands.

Before either existed. `default_threshold(quantity)` — a quarter of current stock, labelled in the UI as a guess. It exists to be accepted and moved past, because §6.7 says an owner must not be asked for a considered decision on each of hundreds of products.

How to build it. The standard reorder-point formula, which is not exotic:

```
reorder point = (average daily demand × lead time) + safety stock
safety stock  = Z × standard deviation of demand during lead time
```

`Z` is the service level: 1.65 for 95%, 2.33 for 99%. For a grocery shop, 95% on staples and lower on slow movers is a reasonable starting point — and the service level is a *business* decision, so it belongs in `app_config` where the owner or the team can change it, not hard-coded.

The work is mostly in getting the standard deviation honest: over the lead-time window, not per day, and excluding stock-out days for the same reason as model 1.

Needs. 8 weeks minimum, 12 for the variance to be stable. Measured lead times per supplier — `supplier_ranking.measured_delivery_days` already holds them, and §12.1 is explicit that the supplier's own

stated `lead_time_days` is a claim, not a measurement.

How it affects the app. The suggestion on `app/(customer)/stocks/add.tsx` and the edit screen changes value, and `threshold_looks_wrong` on the forecast report starts flagging different products. Nothing changes structurally — which is the point of having built the plain version first.

Failure mode to watch. A high safety stock on a perishable product is a instruction to let food rot. The model has no idea what spoils. Either exclude perishable categories or cap the cover days by category; `product_catalog.category` is there.

Model 3 — Learned festival uplift

What the owner gets. "Last Avurudu your sugar sales tripled" instead of "sugar usually rises about 60% at Avurudu". A per-shop, per-product uplift from that shop's own history.

What exists now. `seasonal_events.expected_uplift_pct` — one seeded number per festival for every shop in the country. Avurudu 60%, Christmas 50%, Ramadan 45%, Vesak 30%.

That is wrong in an obvious way, and the stub file says so: a shop in Jaffna and a shop in Galle do not see the same Christmas. A Muslim-majority neighbourhood sees a Ramadan the national average badly understates, and the current number would have them under-order every year.

How to build it. Compare each shop's sales in the window before a past festival against its own baseline for the same products, then store the ratio per shop, per festival, per category. The baseline is the awkward part: the four weeks before the lead-time window is a reasonable choice, but it must exclude any *other* festival's window, and in Sri Lanka the calendar is crowded.

Needs. `MIN_PAST_OCCURRENCES = 1` to say anything at all, and realistically 2 to 3 before it should outrank the seeded estimate. That means **two to three years** for a confident figure on any one festival. This is the model that will be last, and it may never be reached within a student project — which is an honest thing to write in a report rather than a failure to hide.

Also needs maintaining. `seasonal_events.event_date` is the date *for the current year*, and the table's own comment says it needs updating annually because Avurudu, Ramadan and Vesak move. A learned uplift keyed on `name` survives that; one keyed on `event_date` does not.

How it affects the app. `domain/seasonal.suggested_quantity` takes the learned figure when there is one and the seeded estimate otherwise. The dashboard card wording should change with it — "based on your last two Avurudus" is a different claim from "typically", and the owner deserves to know which they are reading.

Model 4 — Supplier recommendation from outcomes

What the owner gets. A supplier suggestion that has learned from what actually happened — who was rejected, who delivered late against their own estimate, who got reordered from — rather than from an assumption that the four weights are right.

What exists now. `domain/ranking.py`: quality 40%, delivery speed 30%, quantity availability 20%, price 10%, with a `NEUTRAL = 0.6` default for a new supplier and rating-count confidence weighting. §12 states plainly that this is not machine learning, and it is the version the app renders.

How to build it. The honest framing is a ranking problem with implicit feedback: each order is an observation that the shop chose supplier A over the alternatives it was shown, and the outcome (delivered on time or not, rated well or not, reordered from or not) is the label. With few orders, the right move is not a learned model but **fitting the existing weights** — using the outcome data to check whether 40/30/20/10 is defensible, and adjusting it. That is a smaller, better-evidenced change than a model, and it is what the data will actually support.

Needs. Completed orders with ratings, not days. The ranking's quality component is 40% and comes entirely from `supplier_ratings` — and **a rating is currently never saved**: `RatingPrompt` collects a score and discards it, and there is no endpoint (`CLAUDE.md` , known gaps). **Model 4 is blocked on that, not on data volume.** Fixing the rating endpoint is a prerequisite and is a small job.

The property that matters. It must stay comparable to the formula. A recommendation that disagrees sharply with `domain/ranking.py` is a signal to check the model, not to trust it. Keep both computable side by side and log the disagreement.

Danger specific to this model. It changes who gets business. A model that quietly down-ranks a supplier on thin evidence costs a real person real income, and the supplier cannot see why. Ranking should stay explainable: an owner should be able to read *why* a supplier was suggested. That is an argument for fitted weights over anything a person cannot read.

7. The difficulties, honestly

These are in rough order of how much damage they do.

Censored demand — the one that bites hardest

If a product is out of stock, it sells zero. That is not zero demand. The model reads the zero as "nobody wanted it", lowers the forecast, lowers the suggested threshold, and so makes the next stock-out more likely. A forecasting model that is fed its own stock-outs will quietly starve the products that sell best.

This is a known problem with a known name, and it is not hard to mitigate here: `stock_adjustments` records every quantity change, so the quantity on hand on any past day can be reconstructed — which `feeds/customer/reports/service.py::_trend` already does for the inventory chart. Days when a product was at zero should be **excluded from the denominator**, not counted as zero sales.

Not yet done in `forecast.py` . It is the first thing to fix.

The data is test data, and shared

Covered in §4. Nobody has used this application as a shop. Until someone does, every number these models produce is a statement about how the team tested, and the team knows that and the reader of a report might not.

Slow products stay unknowable

0.5 sales per product per day across the catalog. Most products will never leave `rough` . Averaging across products to compensate is the wrong answer — it would make a slow product look like a median one. The right answer is per-product confidence and a willingness to say nothing.

A promotion looks exactly like demand

No promotions table. A week of discounted sugar teaches the model that sugar sells well at that rate, and the forecast stays high after the price goes back up. With `pos_sale_items.unit_price` joined in, an unusually low price could at least be *detected*; without it there is no signal at all.

Perishables

Nothing in the data distinguishes rice from milk. A safety stock that is correct for rice is spoilage for milk. `product_catalog.category` is the only handle, and it is a blunt one.

A closed shop and a quiet day are identical

A week's holiday reads as a week of zero demand for everything. `forecast.py` mitigates the obvious case by measuring the window to the **last sale** rather than to today, so a shop closed for a week does not have that week counted against every product — but a closure in the *middle* of the history is still counted as real zeros.

The cold start, which is the whole project's shape

Every model needs history the application only produces once it is being used. §14 of the product overview puts it well: built on purpose to need real sales history first, because a model trained on a week of data is a guess wearing a lab coat. There is no engineering fix for this. The only responses are to refuse to answer, to say how much is missing, and to be useful in the meantime with arithmetic that degrades gracefully — which is what model 1 does.

Nobody can check the answer

There is no held-out truth to score against. Proper evaluation needs a backtest: train on the first N weeks, predict the next, compare. **With 7 days there is nothing to hold out.** At three months a backtest becomes possible and should be the first thing written — until then, "the model works" can only mean "the arithmetic is tested", which is what the 32 unit tests establish and all they establish.

8. Before ML and after, screen by screen

Screen	Before	After
Reports	Five sections, all greyed out saying what was needed first	A forecast section: what runs out when, sorted by urgency, with a confidence label
Stocks list	Quantity and a status badge — thin, but not <i>when</i> it empties	Unchanged by design. The run-out date lives in Reports; \$7.1 keeps reports in the reports feed
Add / edit a product	Threshold suggested as a quarter of current stock, labelled a guess	Suggested from real velocity and the supplier's measured lead time (model 2)
Dashboard festival card	"Sugar usually rises about 60% at Avurudu"	"Your sugar tripled last Avurudu" (model 3)
Supplier list	Ranked on fixed weights 40/30/20/10	Ranked on weights fitted to what actually happened (model 4)
Ordering	Owner decides quantities from memory	A suggested quantity with a reason attached

What does not change, and should not: every write still goes through the backend, the app still renders numbers rather than deriving them, and `domain/` still owns every rule that is not learned. The ML layer is additive. If all four `is_available()` functions returned `False` tomorrow, the application would work exactly as it does today — and that property is worth protecting, because it is what makes a model safe to ship.

9. The honesty rules

These are not decoration. They are the difference between a useful feature and a harmful one, because **a shop owner will order stock against these numbers with their own money.**

1. **Refuse rather than guess.** Below `MIN_DAYS_OF_HISTORY` the report returns no products at all, not a table of zeros. `predict()` returns `None`.
2. **Always say how sure.** `rough` / `fair` / `good`, and `weeks_until_good`. Three words, not a percentage — a confidence interval computed from thirty points would itself be a guess, and "68% ± 12" is neither honest nor actionable.
3. **Never show a date the arithmetic cannot support.** Cover beyond 90 days gives the number and withholds the date.
4. **A learned value never silently replaces a seeded one.** The wording changes with it, so the owner knows which they are reading.
5. **Keep the plain rule next to the learned one.** One `is_available()` returning `False` must be enough to fall back, with no other code change.

10. Architecture

Where code goes. `backend/app/ml/*.py` holds rules only: inputs are plain lists and dicts, outputs are dataclasses, no database and no network. That is what makes them testable without a database, and it is the

same split as `domain/` and `src/pos/*Math.ts`. Fetching belongs in `feeds/customer/reports/service.py`.

Stored or computed? §7.2 requires predictions to be computed on a schedule and stored, so that opening a screen never triggers training. Model 1 is currently **computed on request**, deliberately: it does no training — it is a weighted mean over at most 90 days of one shop's rows, three queries and a loop. Storing it would add a table, a migration and a staleness question to save a few milliseconds.

`ml/storage.py::is_worth_storing()` is the one switch. It returns `False` today. The first model that must be *fitted* rather than counted — model 3 certainly, a weekday effect probably — flips it, and then `jobs/forecast_recalc.py` starts doing its weekly work without changing: it is already registered on the schedule and already calls `storage.refresh_all()`.

When you add the table, its shape depends on what the models output, which is why it does not exist yet. Guessing now means a migration to correct it later.

Dependencies. `scikit-learn` and `numpy` are commented out in `requirements.txt`. Uncomment them when a model genuinely needs them, which is model 3 and possibly model 4 — not before. Models 1 and 2 are standard-library arithmetic, and keeping them that way keeps the deployment small.

11. Testing

- **Unit tests, no database:** `backend/tests/test_ml.py`, run with `pytest` from `backend/`. 32 tests for model 1. Every new model gets the same treatment, and the tests should name the mistake they prevent — `test_a_quiet_day_counts_as_a_zero_not_as_a_missing_day` is more use in six months than `test_rate_calculation`.
- **Against the live database:** `backend/scripts/try_forecast.py` reads the real endpoint as a real shop and prints the table. Scripts in `backend/scripts/` are the place for anything needing a database, and they clean up after themselves.
- **What is still missing:** a backtest. Not possible yet (§7). When it is, it is the only thing that can say whether a model is any good, and it should run as a script with a printed error figure.

12. The order to build in

1. **Exclude stock-out days from model 1.** Correctness, not a new feature. Needs `stock_adjustments` reconstruction, which `reports/service.py::_trend` already demonstrates.
2. **Fix the rating endpoint.** Small, unblocks model 4's only data source, and is a known gap already listed in `CLAUDE.md`.
3. **Decide about seeded history.** Either generate a clearly-labelled realistic dataset so the models can be demonstrated and backtested, or accept that the screens will read `rough` and say why. This is a judgement call about what the project is for, not a technical one.
4. **Model 2**, once there are 8 weeks. The formula is standard; the work is honest variance.
5. **Weekday effect in model 1**, once there are 8 weeks.
6. **A backtest script**, as soon as there is enough history to hold some out.
7. **Model 4** as fitted weights, once ratings and orders exist.

8. **Model 3**, after a festival has been lived through. Possibly never, within this project.

13. Open questions

- **Seeded history: yes or no?** It makes the models demonstrable and makes every screenshot a statement about invented data. If yes, it must be labelled in the data itself, not just in a commit message.
 - **Service level `z` for model 2** — whose decision, and where does it live? `app_config` is the obvious home.
 - **Per-category cover days**, to stop model 2 recommending spoilage?
 - **Does the forecast belong on the Stocks screen as well as in Reports?** §7.1 says reports live in the reports feed. An owner standing at a shelf might disagree.
 - **What happens to a prediction when a product's preferred supplier changes?** The lead time, and so the threshold, changes underneath a stored suggestion.
-

14. Related reading

File	Why
<code>docs/14-product-overview.md</code>	"What is coming next" — the four features in the owner's language
<code>docs/08-api-contract.md</code>	The seasonal warning is not ML and must not become ML
<code>docs/04-decision-log.md</code>	D-023 the owner's till view, D-025 writes only through the backend
<code>docs/16-pos-system.md</code>	Where the till sales come from
<code>CLAUDE.md</code>	The conventions: rules in <code>domain/</code> , fetching in <code>service.py</code> , and the traps
<code>backend/app/ml/forecast.py</code>	The built model, with its reasoning in comments
<code>backend/tests/test_ml.py</code>	What the model is guaranteed to do
<code>database/migrations/0009_sales_records.sql</code>	The training table
<code>database/migrations/0032_reports_can_see_till_sales.sql</code>	Why reports could not see till sales
